OCI distribution (`tfs +oci:`)

TBK SPEC 38:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 38:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword ____ 4

1. Why (the ceiling, again — and the self-hosted tier) ____ 4

2. The reference grammar (spec 04 §1 amendment) ____ 5

3. The artifact model (locked) ____ 7

4. The registry index over OCI (spec 04 §2 / spec 37 §4 amendment) ____ 9

5. The resolve/fetch path (spec 05 §2/§4/§6 amendment) ____ 10

6. The credential chain (spec 37 §5 amendment) ____ 11

7. The publish path (spec 04 §2 / spec 16 §5 amendment) ____ 12

8. Security and trust invariants ____ 13

9. Failure semantics, offline, and the named errors ____ 14

10. Capability gating (spec 04 §3's general rule, applied) ____ 15

11. Migration and coexistence (locked) ____ 15

12. Test plan (the tiers of spec 14; each failure above maps to one tier) ____ 16

13. Open questions (each with this spec's recommendation) ____ 17

Implementation plan (spec PR first; each PR independently mergeable) ____ 18

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Status: SHIPPED — the grammar, the artifact model, and the credential amendments are implemented (`crates/tebako-oci/` + `crates/tebako-resolve/src/oci.rs`); the resolver speaks `tfs+oci://` (`crates/tebako-resolve/src/reference.rs`).

Normative specification of the OCI distribution channel: payload images, runtime bundles, and registry indexes published to any OCI Distribution Spec v1.1 registry (GHCR, Harbor, zot, ECR, ACR, GitLab CR) as generic artifacts, resolved through one new reference class beside spec 04 §1's family. The channel is an ADAPTER — the store layout (spec 05 §3), the trust pipeline (spec 09), and the declarative platform-selection law (spec 04 §2) are untouched.

1. Why (the ceiling, again — and the self-hosted tier)

- **The per-tag asset ceiling.** Spec 36 §1 measured the GitHub release ceiling (1,000 assets per release object; the ruby catalog is ~1,800 per-file assets and forces the line-shard choreography). An OCI registry has no per-release asset concept at all: every artifact is blobs + a manifest, addressed by tag or digest. The catalog publishes flat.
- **The fourth operator tier.** Spec 37 §9's three tiers (static HTTPS, the git host they run, airgap) miss the shop that already runs Harbor/ECR/ACR and wants "our own internal payload instance" off the shelf — with the registry's own RBAC, retention, and replication. OCI is that tier, and it needs no tebako-side service (spec 00 invariant 9 stands: no default service, no server to run).

© 2026 The Tebako Project

- What this never adds: a resolution priority, a default registry host compiled into any binary (docker.io is NOT privileged — a reference names it explicitly or not at all), a silent fallback between channels, a second signature mechanism.

2. The reference grammar (spec 04 §1 amendment)

One new PROTOCOL-class form. The host is ALWAYS explicit — there is no canonical OCI SaaS — so the form is the `tfs+<svc>://host` shape of spec 37 §4, never the bare `tfs:<svc>:` SaaS shape:

FIGURE 1

```
OCI adapter:
  tfs+oci://<registry>[:port]/<repo>[:<tag>|@<digest>][?
sha256=<64 hex>]

  tfs+oci://ghcr.io/tebako-packages/metanorma:1.2.3-linux-
gnu-x86_64
  tfs+oci://harbor.corp.internal/team/metanorma:1.2.3
  tfs+oci://ghcr.io/tamatebako/tebako-runtime-ruby@sha256:
<64 hex>
  tfs+oci://127.0.0.1:5000/fixtures/tool:1.0
(loopback — §8)

DIGEST PINS (two distinct pins, never conflated):
  @sha256:<64 hex>   pins the MANIFEST digest (the OCI-
native immutable form)
  ?sha256=<64 hex>   pins the ARTIFACT bytes (the layer blob
— the same
                     anchor vocabulary every other reference
class uses)
```

ABNF (normative; the OCI distribution-spec name/tag grammars are incorporated, not re-invented):

FIGURE 2

```
oci-ref   = "tfs+oci://" host [ ":" port ] "/" repo
            [ "@" digest | ":" tag ] [ "?" "sha256="
64HEXDIG ]
```


© 2026 The Tebako Project

```
host      = <lowercase DNS name | IPv4 | "[" IPv6 "]" |
"localhost">
repo      = component *( "/" component )
component = [a-z0-9] *( [a-z0-9] | ( "." | "_" | "__" | "-"
) [a-z0-9] )
tag       = [A-Za-z0-9] *( [A-Za-z0-9] | "." | "_" | "-" )
; ≤ 128 chars
digest    = "sha256:" 64HEXDIG
```

- **MECE (locked):** the tag splits at the LAST `:` (a `host:port` never collides — the host came off at the first `/`, exactly the `tfs+github://` rule); `@` selects the digest form; a reference carrying both `:tag` and `@digest` is `Invalid`. Repo names are lowercase-only per the distribution grammar — an uppercase byte in `repo` is a parse-time `Invalid` (65), never a downcasing guess.
- **The bare form `tfs:oci:…`** is the named `Invalid` steering to the hosted spelling: "the OCI form is `tfs+oci://<registry>/<repo>` — there is no canonical OCI host; name the registry explicitly" (the `UnsupportedService` pattern of spec 37 §4: a refusal that teaches, never a guess). docker.io is reachable only as an explicit host (`tfs+oci://registry-1.docker.io/library/…`).
- **No `#artifact` concept.** The tag/digest IS the selector; there is no multi-asset listing to disambiguate (the `AmbiguousAssets` class never fires on this adapter). Repo layout (one repo per payload, one repo per factory line) is publisher policy; the grammar fixes only the tag derivation (§3).
- Both pins may ride one reference. Each is verified fail-closed at its moment: the manifest digest at manifest read (69/70 class below), the byte pin against the layer descriptor BEFORE the stream starts and again inline at end-of-stream (`Sha256Mismatch`, exit 70 — spec 04 §3's discipline, unchanged).

Spec 04 §1's dispatch table gains one row:

TABLE 1

scheme	adapter
`tfs+oci:`	the OCI distribution adapter (this spec)
anything else	**named error listing the classes** (the list gains the OCI form)

A pre-OCI tebako reading a registry whose `release.ref` is `tfs+oci:` fails at parse with the named classes-listing error (65) — loud, never a guess (the additive rule does not cover a PRIMARY ref in an unknown class; that is exactly the spec 36 §4 compat-window behavior).

© 2026 The Tebako Project

3. The artifact model (locked)

One OCI artifact per served file. Every artifact is an OCI image manifest (distribution-spec v1.1 form) with `artifactType` set, the empty config (`application/vnd.oci.empty.v1+json`, the canonical `{}` bytes), and EXACTLY ONE layer — the file's raw bytes. No tar wrapping, no compression: the layer blob IS the file, so the layer digest IS the artifact's sha256 — the `.sha256` sidecar's exact equivalence (spec 05 §2/§4's anchor transfers without a second pin file). A manifest violating the shape (≠1 layer, a foreign `artifactType`, a missing required annotation) is the named `OciArtifactMalformed` (69), never a best-effort read.

TABLE 2

artifact class	`artifactType`	layer media type
payload image (`.tfs`)	`application/vnd.tebako.tfs.v1`	`application/vnd.tebako.tfs.v1+layer`
runtime bundle (spec 36)	`application/vnd.tebako.runtime-bundle.v1`	`application/vnd.tebako.runtime-bundle.v1+tar.gz`
registry index (`tpkg-registry.yaml`)	`application/vnd.tebako.registry.v1`	`application/vnd.tebako.registry.v1+yaml`
detached OpenPGP signature	`application/vnd.tebako.signature.v1`	`application/vnd.tebako.signature.v1+asc`
blksum sidecar (spec 39 §3)	`application/vnd.tebako.blksum.v1`	`application/vnd.tebako.blksum.v1+json`

Annotations mirror resolution fields only (the spec 03 §4 tier-3 rule — the registry mirrors, never duplicates authority). The full L1 manifest stays in-image (tier 1, authoritative); annotations carry the L3 subset, so a dispatcher never pulls a blob to resolve:

TABLE 3

annotation	content
`org.opencontainers.image.title`	the served file name (the shard's `filename` spelling for bundles — flowed, spec 05 §2's SSOT rule)
`org.tebako.name` / `org.tebako.version` / `org.tebako.kind`	the L1 IDENTITY mirror
`org.tebako.triplet`	the host triplet, or `universal`
`org.tebako.entrypoints`	comma-separated names (payloads)
`org.tebako.runtime-requirement`	the requirement object, JSON-encoded (payloads)

© 2026 The Tebako Project

annotation	content
`org.tebako.runtime.shard`	the per-package shard JSON VERBATIM (runtime-bundle artifacts only — one manifest read replaces the shard fetch of spec 36 §4)
`org.tebako.signature.keyid` / `org.tebako.signature.subject`	the signer's PRIMARY keyid (spec 09 §9's primary rule) / the digest of the signed blob (signature artifacts only)
`org.tebako.blksum.subject`	the digest of the image blob the sidecar serves (blksum artifacts only — the sibling the tag is keyed by)

Tag derivation (locked — the single owner of both rules is this section):

- payload: `<version>` for `universal`, `<version>-<triplet>` per triplet (tebako version/triplet characters are a subset of the OCI tag grammar by construction).
- runtime bundle: the bundle STEM verbatim (`tebako-runtime-<ver>-<lang>-<lv>-<triplet>` — spec 36 §2's spelling; the factory declares it, consumers flow it). The OCI lane opened with the post-tebako#716 grammar: pre-#716 (per-file-era) lines never publish to OCI, so a selector-less locator composes the new-era spelling only.
- registry index: `latest` by convention; publishers MAY add version tags; operators pin with the `@sha256:` form.
- signature: `sha256-<64 hex of the SIGNED BLOB's digest>.asc` — keyed by the signed bytes, derived deterministically from the layer descriptor the fetcher already holds. No listing, no guessing.
- blksum sidecar (added with spec 39 §8's OCI arm): `sha256-<64 hex of the IMAGE blob's digest>.blksum.json` — the same sibling digest-tag rule as the signature's, keyed by the image bytes. The artifact's layer digest IS the sidecar document's own sha256 — the seed descriptor's `blksum_sha256` pin (spec 39 §3) binds it with no second file.

Per-triplet tags, never an OCI image index (locked — the tebako#695 DECIDE). Platform selection in tebako is the REGISTRY's declarative job (spec 04 §2, locked: "the adapter NEVER auto-picks by host triplet"). An OCI index manifest would import a second, adapter-side platform matcher — and its `os/arch` vocabulary cannot express tebako triplets (`linux-gnu` vs `linux-musl` is a libc axis OCI has no word for; spec 05 §5's wheel-tag table rides the triplet axis alone for the same reason). One artifact per image under a per-triplet tag keeps the declarative law absolute, keeps byte-identity with the GitHub-releases artifact trivially auditable (same blob, same digest), and keeps the payload case and the runtime-pair case on ONE shape. A future multi-file need ships as a new artifact CLASS, never a second layer meaning and never an index.

Signatures ride sibling digest-tags, not the referrers API (locked). The v1.1 referrers API is the semantically rich form, but the issue's target set (ECR, ACR, Harbor, zot) is exactly where referrers coverage is uneven in 2026, and a referrers-required design would re-introduce per-registry capability negotiation — a silent capability matrix, the opposite of named errors. The sibling-tag form is distribution-spec v1.0-clean, deterministic, and preserves spec 09 §5's every-served-name rule literally: the served name (the tag) has its own `.asc` artifact, one signature covers exactly one artifact, nothing is ever "covered by" a container's signature. Referrers MAY be added later as an additive MIRROR (publish

© 2026 The Tebako Project

both); resolvers never require it. For `tfs+oci` release refs the registry row's `signature.asc` locator is DERIVED by the tag rule — an authored `asc` field on such a row is a named registry validation error (a locator that would be ignored is an authoring bug), while `signature.keyid` pins identity exactly as today.

4. The registry index over OCI (spec 04 §2 / spec 37 §4 amendment)

A `tpkg-registry.yaml` published as a §3 registry-class artifact is a REGISTRY LOCATION — the third location form, beside the git-host forms and spec 37 §4's `tfs+https://` registry file:

FIGURE 3

```
tebako add-registry tfs+oci://harbor.corp.internal/team/tpkg-
registry[:tag]
tebako add-registry tfs+oci://harbor.corp.internal/team/tpkg-
registry@sha256:…
```

- The alias derivation, the book model, `require_signed`, the official seed — all of spec 37 §2 applies unchanged (the alias derives from the ref's repo component).
- TTL mapping (locked): a TAG-pulled OCI registry rides the dispatch-time cache exactly like the git-host default-branch form — `registries/<sha>.yaml` + `.fetched-at`, 24 h TTL, spec 05 §4's stale-serve on refresh failure (the manifest digest is recorded in the cache sidecar; a refresh that resolves the same digest re-serves the cached bytes, saving the blob pull). A DIGEST-pulled OCI registry is the pinned-immutable form — the analog of `tfs:<svc>:owner/repo:version#tpkg-registry.yaml`: cached forever, never re-fetched, TTL inapplicable.
- `TEBAKO_OFFLINE=1`: cached-within-TTL serves; anything else is the named offline error. Unchanged semantics, new adapter.

© 2026 The Tebako Project

5. The resolve/fetch path (spec 05 §2/§4/§6 amendment)

Manifest reads (and the shard/registry-YAML blobs they name) join the "small buffered GETs" class; the artifact blob is a spec 05 §6 FetchPlan ITEM — the pipeline owns transport, integrity, scheduling:

1. **Manifest read.** GET `/v2/<repo>/manifests/<tag|digest>` with the §6 credential decision; a `?sha256=` byte pin is checked against the layer descriptor digest BEFORE the blob stream starts (a fail-fast 70, never a wasted pull); an `@sha256:` reference verifies the returned manifest against the pin (the registry's `Docker-Content-Digest` and the body digest must agree — 70 class).
2. **Blob pull.** GET `/v2/<repo>/blobs/<digest>` streams through the pipeline's HashWriter — one pass, constant memory, the inline sha256 compared against the layer digest AND any byte pin. The layer digest is the registry-supplied trust anchor; the store's `.sha256` marker is written from it, byte-identical in shape to today's sidecars.
3. **Commit.** The commit closure owns install semantics exactly as today: the store's per-entry flock (120 s, stale-lock hint), tmp + rename, the spec 05 §4 commit order (the manifest mirror is the commit point — a version whose mirror is absent was never installed), signature verification per spec 09 BEFORE the bytes enter the cache. A partial pull is invisible by the same proof.
4. **Runtime bundles.** A `kind: runtime` row whose `release.ref` is `tfs+oci:` resolves through spec 37 §8's chain: the locator derivation (`release_download_locator`) gains the OCI arm, and the fetch runs the bundle flow below. OCI runtime distribution is bundle-era only: per-file-era lines stay on their git-host releases forever (keep-forever, spec 13 §8) and never publish to OCI.
   - **The fetch tag.** A selector-less locator tags the bundle STEM (`tebako-runtime-<ver>-<lang>-<lv>-<triplet>` — §3's tag rule, the factory's spelling flowed verbatim; the OCI lane is post-tebako#716, so only the language-carrying spelling exists here); an authored `:tag` or `@sha256:` selector rides exactly as written (spec 37 §8's authored-selector law).
   - **One manifest read replaces the shard fetch.** The per-package shard comes from the bundle manifest's `org.tebako.runtime.shard` annotation (§3) and is gated THREE ways before any blob streams: the shard's identity triple (`<engine>_version` / `tebako_version` / `platform`) must match the runtime being resolved — a mismatch means the tag resolved another runtime's publish (69); the shard MUST declare a `bundle` block — a per-file-era shard over OCI is the named bundle-era-only refusal (69), never a per-file fallback; and the layer descriptor digest — which IS the bundle's `.sha256` sidecar (§3) — must agree with the shard's bundle pin (a disagreement is 70, nothing installed).
   - **The plan re-resolves by digest.** The FetchPlan item re-reads the ALREADY-RESOLVED manifest by its digest — the tag's mutability never reaches the stream — and streams the one bundle layer under the RuntimeBundle shape law; spec 36 §4 then runs VERBATIM (the signature verified FIRST — the `.asc` sibling digest-tag derived from the signed blob's digest, a declared-but-unfetchable signature refused

© 2026 The Tebako Project

71 exactly like the git-host lane — then bundle digest → in-process unpack → per-member pins → the spec 05 §3 layout). The staged store entry is indistinguishable from a git-host bundle install, the origin markers excepted (point 5's digest-pinned form).

- — **The lazy mode degrades loud.** OCI serves the bundle whole — there is no per-file arm and no blksum probe on this channel — so the lazy mount mode against an OCI source is the loud eager fallback (`event=lazy-fallback reason=oci-bundle-era-only`) and the install completes eager.
- — **`channel: oci` covers runtime rows.** A book entry declaring `channel: oci` (§11) routes the picked runtime row through its per-triplet `oci:` mirror field — fail-closed exactly like the payload lane: a universal row, a host-covered row lacking the mirror, or an unparsable one is a named 65 error naming the registry and the row; never a fallback to the primary `release.ref`.

5. **The origin marker** records the concrete digest-pinned form (`tfs+oci://<host>/<repo>@sha256:<manifest digest>`) even for tag pulls — spec 37 §7's origin binding and `tebako cache list` work unchanged, channel included.

The retry/throttle law is tebako-http's, per worker per connection, unchanged; the plan-cancel failure law is spec 05 §6's, unchanged (first named error surfaces, every tmp dropped, nothing partial in the store).

6. The credential chain (spec 37 §5 amendment)

The two-tier book is untouched in shape; the OCI adapter adds one fallback tier and the distribution-spec token flow:

1. **Tier 1** — the directing registry's alias, confinement included (the tier-1 `allowed_hosts` set for an OCI registry is its registry host; derivation rides the same base-URL construction SSOT).
2. **Tier 2** — the URL's exact host.
3. **The docker/config.json fallback (NEW, OCI-class URLs only):** static `auths[<host>]` entries (the base64 `user:pass` pair) from `~/.docker/config.json` (plus `$DOCKER_CONFIG`), journaled as `token:docker-config:<host>`. `credHelpers`/`credsStore` entries covering a referenced host are the named `DockerCredentialHelperUnsupported` (65) — helpers are shell-outs, forbidden by the no-shell-outs law, and the steer says so (move the token into a `credentials:` entry's env var). An unparsesable docker config is `DockerConfigMalformed` (65), never a silent skip.
4. **Anonymous.**

© 2026 The Tebako Project

- **The Bearer token flow (normative):** every registry call runs the distribution-spec challenge dance — on 401 with `WWW-Authenticate: Bearer realm=…, service=…, scope=…`, GET the realm with the basic credential (`user:password`; the `token_env` value MUST carry the pair — a value without `:` is `CredentialRequired` naming the expectation, never a username guess), then ride the returned bearer. Anonymous pulls challenge identically without credentials (GHCR's anonymous-token shape). Tokens cache per process keyed by (realm, service, scope, credential class); a 401 against a cached token re-challenges ONCE, then fails named. A malformed or Basic-only challenge is `OciTokenChallengeInvalid` (69).
- **Confinement (locked, spec 37 §5's rule extended):** the basic credential is presented to the challenge REALM via the ordinary book lookup on the realm URL's host — a realm whose host is not the registry host matches its own tier-2 entry or is the named `OciCrossHostAuthRefused` (69) steering the operator to add one (this covers docker.io's `auth.docker.io` with zero special cases and zero hardcoded hosts). The returned bearer rides ONLY to the registry host — never cross-host, never to a redirect target (ureq's `RedirectAuthHeaders::Never` stands).
- The header shape by service class gains the OCI row: `Authorization: Basic <base64(user:pass)>` to the realm, `Authorization: Bearer <token>` to the registry — the adapter owns the shape; authored config never spells header mechanics.
- Every fetch journals host + credential class, values redacted — spec 37 §5's audit trail, new classes (`token:docker-config:<host>`) included.

7. The publish path (spec 04 §2 / spec 16 §5 amendment)

The normative publisher is `tebako publish` — the OCI leg is in-process (blob mount attempts first, monolithic POST+PUT upload, manifest PUT), never the ORAS CLI (the no-shell-outs law binds shipped artifacts and tests; ORAS in a CI workflow is permitted sugar, never the contract).

- `tebako publish … --oci tfs+oci://<host>/<repo>` uploads each payload's §3 artifact (plus signature artifacts when signing, plus the blksum sidecar sibling for every payload the staged sidecar exists for — spec 39 §8) from the SAME staged bytes the

© 2026 The Tebako Project

git-host release leg uploads, and writes the additive `oci:` mirror field into the registry row it maintains:

FIGURE 4

```
platforms:
  x86_64-linux-gnu:
    artifact: metanorma-1.2.3-linux-gnu-x86_64.tfs
    sha256: "…"
    oci: tfs+oci://ghcr.io/tebako-packages/
metanorma:1.2.3-linux-gnu-x86_64
```

- **Write-once (the release-immutability rule transferred, spec 13 §2a):** publish refuses to point an existing tag at a DIFFERENT digest — `OciTagConflict` (69) naming the tag and both digests. Re-publish of identical bytes is the idempotent skip, as today. Tags are mutable in the OCI model; tebako treats them as immutable by policy, and withdrawal stays `status: withdrawn` (spec 04 §2), never a tag delete.
- **Feedstock adoption is additive and non-disruptive:** the publish invocation gains the flag; without it, bytes, rows, and workflows are exactly today's. Pre-OCI readers ignore the `oci:` field (spec 37 §2's forward-compat leniency for unknown registry keys).
- Runtime factories: the tebako-release publish leg pushes the §3 bundle artifact + shard annotation + signature artifact per leg on bundle-era lines (spec 36 §6's topology unchanged — the OCI push is the same invocation's second sink, and the line-shard escape hatch is RETIRED on OCI-published lines: there is no ceiling to shard against). The factory-side wiring lives in the factory repos; the wire shape is §3's and lands with the product PRs.

8. Security and trust invariants

- **HTTPS-only (locked).** A non-`https` OCI URL fails at parse/fetch with `OciInsecureTransport` (65) — there is no insecure-registry spelling and there never will be (spec 04 §4's no-verify-off law). The single carve-out is LOOPBACK (`localhost`, `127.0.0.0/8`, `::1`) over plain HTTP, existing for the §9 test fixture and local development; it can never name a remote host.
- **TLS roots ride tebako-http's one agent** (spec 04 §4): webpki-roots default, `TEBAKO_TLS_PLATFORM_ROOTS`, additive `extra_ca` — internal registries under an enterprise CA are the `extra_ca` case, and the proxy policy (CONNECT, 407 naming) applies to token and blob calls alike. No second client anywhere in the adapter.

© 2026 The Tebako Project

- **Signature strictness is spec 09's, unchanged:** a declared signature verifies strict (71 invalid / 72 untrusted keyid, the primary-keyid resolution of spec 09 §9 included); a declaration whose `.asc` artifact does not fetch is the invalid-signing-state 71; unsigned fetches warn loud + journal and fail closed under `TEBAKO_REQUIRE_SIGNED=1`; `require_signed` on the book entry binds OCI rows exactly as git-host rows (spec 37 §2.2). The store markers keep their semantics — verification at fetch/install, never per run.
- **Trust-anchor equivalence (locked):** the layer digest IS the artifact sha256. A registry row's `sha256` pin, a `?sha256=` reference pin, and the layer descriptor must agree or the fetch fails (70) — three spellings of one anchor, never three anchors.

9. Failure semantics, offline, and the named errors

`TEBAKO_OFFLINE=1` is cache-or-named-error per adapter: cached store entries serve; any manifest/tag resolution is the named offline refusal. The plan-cancel law (spec 05 §6) makes every partial pull — manifest half-read, blob half-streamed, signature missing — invisible: tmp files dropped, nothing renamed, the first named error surfaced.

TABLE 4

Name	Exit class	Moment
`Invalid` (re-scoped; OCI reasons)	65 usage	parse: uppercase repo, bad tag/digest, `:tag+@digest`, bare `tfs:oci:` (steers to the hosted form)
`OciInsecureTransport`	65	non-HTTPS registry URL off loopback
`DockerConfigMalformed`	65	the docker-config fallback consulted and unparsesable
`DockerCredentialHelperUnsupported`	65	a `credHelpers`/`credsStore` entry would answer — the no-shell-outs steer
`OciManifestNotFound` / `OciBlobUnknown`	69 unavailable	`NAME_UNKNOWN` / `MANIFEST_UNKNOWN` / `BLOB_UNKNOWN` / `TAG_INVALID` mapped by name
`OciArtifactMalformed`	69	§3 shape violation — ≠1 layer, foreign `artifactType`, missing required annotation
`OciTokenChallengeInvalid`	69	malformed or Basic-only `WWW-Authenticate`; unusable token endpoint

© 2026 The Tebako Project

Name	Exit class	Moment
`OciCrossHostAuthRefused`	69	challenge realm host ≠ registry host and no book entry covers the realm — steer: add a tier-2 `host:` entry
`CredentialRequired` (reused)	69	401/403 with no usable credential — names registry + env var looked for
`OciAdapterDisabled`	69	`tfs+oci:` reached a build with the `oci` feature off (§10)
`OciTagConflict`	69	publish: tag exists at a different digest — write-once
`Sha256Mismatch` (reused)	70 integrity	pin/layer/bytes disagreement — nothing cached
spec 09 classes	71 / 72	signature invalid / signer untrusted / declared `.asc` unfetchable

Each maps to exactly one CI tier's assertion; none is a fallback.

10. Capability gating (spec 04 §3's general rule, applied)

The adapter is a cargo feature `oci` on tebako-resolve (backing the new `tebako-oci` client crate) — default ON for the toolchain (tebako CLI, tebako-shim, tfs CLI), OFF for the size-gated tebako-bootstrap, where a `tfs+oci:` reference fails closed with the named `OciAdapterDisabled` (the `GitAdapterDisabled` precedent: steer to managed mode). The bootstrap's own runtime base stays the spec 05 §2 default + `TEBAKO_RUNTIME_MIRROR` (https/file) — OCI runtime distribution enters through the toolchain's registry-derived channel, and the 3 MiB gate never pays for it.

11. Migration and coexistence (locked)

- **Dual publish is the norm during migration:** the same bytes ride the git-host release AND the OCI repo from one publish invocation; the row carries both locators. The `sha256` pin binds both — a mirror serving different bytes is a 70, so the channels can never silently diverge.
- **Channel selection is DECLARED, never probed.** The primary `release.ref` resolves by default (today's behavior, unchanged bit for bit). A registry-book entry MAY declare

© 2026 The Tebako Project

`channel: oci` (spec 37 §2's config model): rows then resolve through their `oci:` field, and a row lacking one is a named error naming the row — fail-closed, never a fallback to the primary. There is no both-fetch, no race-the-channels, no failover.

- **The ambiguity law is untouched.** A payload name in an OCI-hosted registry AND a git-host registry is `AmbiguousRegistries` exactly as spec 37 §3 (the `oci:` row field is not a registry and adds no ambiguity axis); origin binding (spec 37 §7) confines version chains to the resolving registry and records the channel in the origin marker; a rebind across channels follows the same explicit reinstall

  — journaled `origin-rebind` discipline.

12. Test plan (the tiers of spec 14; each failure above maps to one tier)

- **Unit** (tebako-resolve): the §2 grammar — round-trip parse/display for every form (tag, digest, both pins, port, loopback, IPv6), every malformed class by name; tag and signature-tag derivation; the §3 annotation map; challenge parsing; docker-config parsing and the helper refusal; the credential chain order (tier-1 confinement into OCI hosts, realm-host lookup).
- **Property** (proptest, the crate's existing harness): arbitrary valid references round-trip; arbitrary garbage never panics and never parses silently (a named error, always); tag derivation injective per (version, triplet).
- **Contract** (the Transport mock, canned distribution-API answers): challenge→token→manifest→blob happy path; the error-code map (`NAME_UNKNOWN`→`OciManifestNotFound`, …); media-type/shape refusals; cross-host realm refusal; token re-challenge-once; store-equivalence — an install from OCI and from `file://` of the SAME bytes yields identical store records except the origin marker (the parity arm of spec 00 invariant 8).
- **E2e** (`ci/`, tiered): a **zot fixture** — a pinned zot binary (CI tooling, fetched at workflow time, never a shipped dependency) configured in-tmp on `127.0.0.1:<port>` over plain HTTP, exercising exactly the §8 loopback carve-out. Legs: publish payload + signature → `add-registry` over OCI → install → run the entrypoint; digest-pinned pull; offline refusal; `TEBAKO_REQUIRE_SIGNED=1` pass/fail; tag write-once conflict on re-publish; (with spec 36) the runtime-bundle leg — factory-shaped bundle push → dispatch-time runtime fetch → run. A nightly live-GHCR smoke leg mirrors the existing live-GitHub legs.

© 2026 The Tebako Project

13. Open questions (each with this spec's recommendation)

1. The issue's `tfs:oci:` sketch vs `tfs+oci://`. Recommended and locked: `tfs+oci://` — spec 37 §4 puts every explicit host in the `tfs+<svc>://` shape, and `tfs:<svc>:` names a canonical SaaS host, which OCI deliberately lacks. The bare form is the teaching refusal.
2. Referrers API for signatures. Recommended: sibling digest-tags (locked, §3); referrers as a later additive mirror once ECR/ACR/ Harbor coverage is uniform. Revisit no earlier than the first registry-conformance survey.
3. docker `credHelpers`/`credsStore`. Recommended and locked: refuse by name (65). Helpers are process shell-outs; invariant 1 admits no exception, and the env-indirection book already expresses every static case.
4. Plain-HTTP internal registries. Recommended and locked: refuse; loopback only. Internal CAs are the `extra_ca` feature's case (spec 04 §4) — an insecure-registry knob would be the verify-off spelling that spec bans.
5. `?sha256=` on `tfs+oci:` references. Recommended and locked: pins the ARTIFACT bytes (the layer blob), keeping the cross-class anchor vocabulary; `@sha256:` pins the manifest. Both fail-closed.
6. Chunked/resumable blob upload. Recommended: monolithic POST+PUT in v1 (bundles are ≲150 MB; tebako-http already carries `UPLOAD_TIMEOUT`); chunked upload lands only when a measured publish needs it.
7. OCI as `TEBAKO_RUNTIME_MIRROR`. Recommended: out of scope. The bootstrap keeps https/file bases (§10); OCI runtime distribution enters through the registry-derived toolchain channel.
8. Tag mutability. Recommended and locked: write-once by publish policy (`OciTagConflict`); the digest form is the pin; withdrawal stays `status: withdrawn`, never a tag delete.
9. Image indexes for multi-triplet artifacts. Recommended and locked: rejected (§3) — platform selection is the registry's declarative law; OCI's os/arch vocabulary cannot speak tebako triplets.
10. One repo per payload vs shared repos. Recommended: publisher policy (one repo per payload name is the convention); the grammar fixes only tag derivation, so no resolver behavior depends on the choice.

© 2026 The Tebako Project

Implementation plan (spec PR first; each PR independently mergeable)

1. PR 1 — the spec. This document + `00-INDEX.md` entry + the registry JSON Schema's additive `oci:` field. No code.
2. PR 2 — the pull path, end-to-end. `tebako-oci` crate (model + auth + client), `Reference::Oci` grammar, fetch/pipeline integration, credential chain (docker fallback, realm confinement), registry-YAML-over-OCI (`add-registry`), the `oci` cargo feature + `OciAdapterDisabled`, and the full unit/property/contract tier + zot e2e pull legs. Readers accept and resolve `oci:` row fields (primary-ref resolution only; `channel:` declaration not yet).
3. PR 3 — the publish path + mirror coexistence. `tebako publish --oci` (artifacts, signature artifacts, write-once `OciTagConflict`, `oci:` row mirror write), the book's `channel: oci` declaration with fail-closed row errors, zot e2e publish→install→run legs, tebako.org tier-4 how-to (separate repo).
4. PR 4 — runtime bundles over OCI. Shard annotations, the `release_download_locator` OCI arm, dispatch-time runtime fetch through spec 37 §8's chain, e2e bundle legs. Sequenced with spec 36's landing; if 36 slips, PRs 1–3 still deliver the full payload/registry value of the issue.

© 2026 The Tebako Project